

BRAC'T's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 1

Roll no: 22 PRN No: 12414149 Name of Student: Prathistha Yadav

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Import the required libraries: TensorFlow and Matplotlib.
2. Load the Fashion MNIST dataset and unpack it into training and testing image and label tuples.
3. Normalize the image pixel values for both the training and testing sets by dividing them by 255.0.
4. Add a Flatten layer to reshape the input images from 28x28 to a 1D array.
5. Add a Dense hidden layer with 128 units and a Rectified Linear Unit (ReLU) activation function to learn complex data patterns.
6. Add a Dense output layer with 10 units and a softmax activation function for multi-class classification.
7. Compile the model using the Adaptive Moment Estimation ('adam') optimizer, 'sparse_categorical_crossentropy' as the loss function, and track the 'accuracy' metric.
8. Train the compiled model on the training images and labels for 3 epochs.
9. Evaluate the trained model's performance on the testing set to calculate validation loss and accuracy.
10. Save the final model as a file named fashion_mnist_model.keras.

Code :

```
# Import libraries
import tensorflow as tf
import matplotlib.pyplot as plt

# Load dataset
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.fashion_mnist.load_data()

# Normalize dataset
train_images = train_images/255.0
test_images = test_images/255.0

# Print shapes and display first image
print(train_images.shape)
print(test_images.shape)
print(train_labels)

plt.imshow(train_images[0], cmap='gray')
plt.show()

# Build and compile the model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
my_model.fit(train_images, train_labels, epochs=3)

# Evaluate the model
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print("Test accuracy : ", val_acc)

# Save the model
my_model.save('fashion_mnist_model.keras')
```

Output :

```
[2] import tensorflow as tf
import matplotlib.pyplot as plt

[5] (train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 0s 0us/step

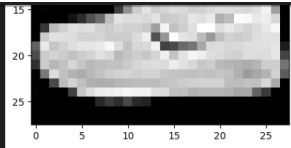
[6] train_images = train_images/255.0
test_images = test_images/255.0

[9] print(train_images.shape)
print(test_images.shape)
print(train_labels)

plt.imshow(train_images[0], cmap='gray')
plt.show()

... (60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]

... 
```



```
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu')) #Rectified Linear Unit #Dense layers are used to learn complex patterns in data.
my_model.add(tf.keras.layers.Dense(10, activation='softmax')) #softmax is often used as the activation function in the output layer for classification tasks

my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive Moment Estimation #sparse_categorical_crossentropy for multi-class classification
my_model.fit(train_images, train_labels, epochs=3)
```

[10]

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(**kwargs)
Epoch 1/3
1875/1875 ----- 8s 4ms/step - accuracy: 0.8247 - loss: 0.4999
Epoch 2/3
1875/1875 ----- 7s 4ms/step - accuracy: 0.8637 - loss: 0.3774
Epoch 3/3
1875/1875 ----- 8s 4ms/step - accuracy: 0.8763 - loss: 0.3406
... <keras.src.callbacks.history.History at 0x7c49514c8da0>
```

```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print("Test accuracy : ", val_acc)
```

[12]

```
... 313/313 ----- 1s 2ms/step - accuracy: 0.8645 - loss: 0.3800
Test accuracy : 0.8644999861717224
```

```
my_model.save('fashion_mnist_model.keras')
```

[14]